CS614 - Midsem Project Review

Aditi Khandelia **Arush Upadhyaya** **Kushagra Srivastava** **Sankalp Mittal** **Vidhi Jain**

March 17, 2026

Project Overview

Iterative pre-copy live migration transfers memory pages from a running process to a destination host while the process continues to execute, repeating the transfer for pages that were dirtied during the previous round until the remaining working set is small enough to complete migration with a brief final pause. The efficiency of this scheme depends entirely on the ability to identify, at the end of each round, *exactly* which pages were written. For CPU memory, the Linux kernel provides this capability through soft-dirty bits embedded in page-table entries, readable via `/proc/PID/pagemap`. No analogous mechanism exists for GPU-managed memory. NVIDIA's Unified Virtual Memory (UVM) subsystem, which underpins `cudaMallocManaged` and handles demand-paging between host and device, exposes no interface for querying which pages a process has modified. Consequently, any migration or checkpointing system must conservatively copy the *entire* GPU memory footprint on every iteration, rendering iterative pre-copy migration of large GPU workloads impractical.

This project addresses that gap by implementing **per-process dirty page tracking** directly within the open-source NVIDIA UVM kernel module. UVM already intercepts every GPU page fault to manage demand-paging; we augment the write-fault servicing path to record each written page in an in-kernel data structure, without introducing any additional hardware overhead. A `procfs`-based interface allows a privileged migration daemon to start and stop tracking epochs and to retrieve the complete set of pages written by a given process, with **page-level granularity** and a per-page **nanosecond-resolution timestamp** suitable for ordering events across migration rounds.

Group Details

Following are the details of the group members:

Name	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in
Sankalp Mittal	220963	sankalpm22@iitk.ac.in
Vidhi Jain	221191	vidhijain22@iitk.ac.in

Milestones

Milestone 1 - Midsem Review ***(4 weeks)***

1. **Literature Review, Technical Familiarization and other Preliminaries** *(1 week)*
   (a) Review existing technical documentation and architectural specifications regarding UVM.
   (b) Conduct a code review of the UVM driver and perform preliminary API experimentation.
2. **Implementation of Dirty Access Tracking in UVM Driver** *(2 weeks)*
   (a) Add instrumentation and logging to observe runtime behavior, including tracking page faults, permission transitions, migration events, and residency changes for debugging and analysis.

(b) Implement the core dirty access tracking mechanism, including data structures for tracking dirty pages, and the logic to update these structures on write faults.

(c) Develop a `procfs`-based interface to allow user-space tools to query the set of dirty pages for a given process, including timestamps for each page.

3. **Initial Performance Evaluation** *(1 week)*

(a) Evaluate the performance of the modified driver on small workloads, measuring metrics such as page fault latency, migration throughput, and overall application runtime.

(b) Identify and address any significant performance bottlenecks or correctness issues in the implementation.

Milestone 2 - Extensions, Testing and Optimisations ***(4 weeks)***

1. **Extension of Functionality** *(2 week)*

(a) Extend the tracking mechanism to support additional features such as multiple concurrent processes, concurrent tracking on CPU and GPU side, and removing write permission for the tracked pages at init, instead of invalidating them to reduce overread due to read-only GPU pages and using kernel threads to parallelly do the same to achieve speedup

(b) Extend the `procfs` interface to expose richer statistics, including per-page access frequency counters and dirty page age, to enable more informed migration and eviction decisions by user-space tools.

2. **Testing and Analysis of Tracking Mechanism** *(1 week)*

(a) Test critical issues pertaining to concurrency and performance that arise under multi-process workloads, including correctness of dirty page tracking under concurrent write faults and race conditions in shared data structures.

(b) Quantify overhead introduced by dirty tracking, measuring additional latency per write fault due to metadata updates, and memory footprint of tracking data structures such as counters, hash tables, and bitmaps.

(c) Profile driver performance across diverse workloads with and without dirty tracking enabled, for metrics such as page fault latency, migration throughput, and overall application runtime.

3. **Optimisation of Tracking Mechanism** *(1 week)*

(a) Based on the analysis, identify and implement optimizations to reduce overhead, such as batching updates to tracking data structures, using more efficient data structures (e.g., bitmaps instead of hash tables), and optimizing critical paths in the tracking logic.

Milestone 3 - Final Deliverables and Integration ***(2 weeks)***

1. Prepare comprehensive documentation for the tool, including usage guidelines and performance considerations.
2. Produce formal benchmarking results quantifying the tracker's performance impact across various scenarios.
3. Implement user-facing configurability for the tracker via a command-line interface (CLI).

Source Code

Development Environment

Development and testing are conducted on a dedicated virtual machine (`dirtyuvm`) running:

- Linux kernel version 6.8 (custom build)
- Modified NVIDIA UVM driver version 580.65.06 (loaded as a kernel module)

Repository

The full source code is hosted at:

`https://github.com/aleph-5/open-gpu-kernel-modules`

This is a fork of NVIDIA's open-source GPU kernel modules, with our modifications applied on top.

Modified Driver Files

All driver modifications reside under `kernel-open/nvidia-uvm/`. The files containing functionality added or significantly modified by us are:

1. `uvm_common.h` / `uvm_common.c` - The primary location of our implementation. Defines `struct dirty_page_info` (page number, timestamp, PID) and `struct uvm_dirty_page_table` (an `xarray`-based per-process table). Implements the core API: table initialisation and teardown, fault recording, and page lookup. Also declares `uvm_dirty_procfs_init` for the `procfs` interface.
2. `uvm_va_block.c` - Hooked to invoke the dirty tracking record function when a write fault is serviced on a VA block.
3. `uvm_gpu_replayable_faults.c` - Modified to record dirty page events when GPU write faults are replayed.
4. `uvm_va_space.c` / `uvm_va_space.h` - Extended to register the dirty-page GPU-unmap invalidation callback (`uvm_va_space_dirty_init`) and manage per-process tracking table lifetime alongside VA space creation and destruction.
5. `uvm.c` - Top-level integration point; initialises the procfs dirty tracking interface at module load.

Tests

User-space tests are categorized into correctness and performance tests and located in the `correctness_tests/` and `performance_tests/` directories respectively

The correctness tests are run everytime the driver is modified and reloaded using `run_regression.sh`

Current Progress

Implementation of Dirty Access Tracking in UVM Driver

Workflow A tracking epoch begins when a privileged user-space tool writes to `dirty_pids_start_track`. This initialises a fresh per-process table and immediately invalidates all current GPU page-table entries by unmapping them, so that every subsequent GPU access generates a fresh page fault. Each write fault is intercepted in the driver's fault-servicing path, which records the faulting page in the table along with a nanosecond-resolution timestamp before re-establishing the mapping and replaying the faulting instruction. At any point during the epoch the tool can read `dirty_pages` (provided that the PID has been set using `dirty_pid_to_query`) to obtain the accumulated set of written pages; reads are non-destructive, so the table is not consumed by the query. The epoch ends when the tool writes to `dirty_pids_stop_track`, which frees all recorded state. Re-starting tracking destroys any existing table before creating a fresh one, cleanly beginning a new epoch.

Data Structures The tracking state is organised as a two-level structure defined in `uvm_common.h`.

1. `struct dirty_page_info` - a per-page record holding the page number, the `ktime_get_ns()` timestamp of the first observed write fault, and the PID of the faulting process.
2. `struct uvm_dirty_page_table` - a per-process container holding an `xarray` that maps page numbers to `dirty_page_info` pointers.
3. A global `xarray` (`pid_to_page_table`) maps each tracked process's `tgid` to its `uvm_dirty_page_table`.

This two-level design provides natural per-process isolation. Entries are recorded under a *first-write-wins* policy: an existing entry is never overwritten, so the stored timestamp always reflects the earliest observed write to that page within the current epoch. The faults may be serviced by different CPU-side driver threads, thus the timestamp may not be strictly monotonic, depending upon the driver-side service thread recording the same.

APIs Exposed User-space interacts with the tracker exclusively through five `procfs` entries created under `/proc/driver/nvidia-uvm/`:

1. `dirty_pids_start_track` (write) - initialises (or re-initialises) the tracking table for `current->tgid` and triggers GPU-mapping invalidation to start a new epoch.
2. `dirty_pids_stop_track` (write) - destroys the table for `current->tgid`, freeing all recorded entries.
3. `dirty_pid_to_query` (write) - sets the PID whose table is consulted on subsequent `dirty_pages` reads, decoupling the querying process from the tracked process.

4. `dirty_pages` (read) - iterates the selected process's table and emits one line per recorded page in the format `0x`*`addr timestamp_ns pid`*. Reads are *non-destructive*: entries remain in the table and can be queried repeatedly within the same epoch.
5. `dirty_range` (write) - accepts a pair of hexadecimal addresses `0x`*`start`* `0x`*`end`* (end exclusive) to restrict subsequent `dirty_pages` reads to a specific virtual address window. The range is protected by a spinlock to allow concurrent updates safely. An inverted or zero-length range produces an empty result rather than undefined behaviour.

Internal Functions and Implementation Sites The implementation is spread across several driver files, each responsible for a distinct concern:

1. `uvm_common.c` - the primary implementation file. Contains `uvm_dirty_page_table_init` / `uvm_dirty_page_table_destroy` (table lifecycle), `uvm_dirty_page_table_record` (fault recording, using `GFP_ATOMIC` as it executes in a non-sleepable context), `uvm_dirty_page_table_lookup`, and the full `procfs` handler implementations including `uvm_dirty_procfs_init`.
2. `uvm_gpu_replayable_faults.c` - the write-fault hook. After standard fault servicing, checks `uvm_dirty_tracking_active_for_pid(va_block->creator_pid)` and calls `uvm_dirty_page_table_record` for `UVM_FAULT_ACCESS_TYPE_WRITE` faults.
3. `uvm_va_space.c` - implements `uvm_dirty_invalidate_all_gpu_mappings`, which walks all live VA spaces and blocks to unmap GPU PTEs at epoch start. Exposes `uvm_va_space_dirty_init` to register this function via the `uvm_dirty_invalidate_fn` function pointer at module load.
4. `uvm.c` - top-level integration; calls `uvm_dirty_procfs_init` and `uvm_va_space_dirty_init` during module initialisation.

Correctness Tests

At this stage, evaluation is driven by the CUDA tests under `correctness_tests/`. The suites are intended to cover four broad questions:

1. **Generic sanity checks:** The `generic` suite is the basic end-to-end test for dirty-page tracking: it GPU writes are recorded, read-only accesses are not, and that reset, disable, PID reporting, and range filtering behave as expected.
2. **Table lifecycle:** The `table_lifecycle` suite checks whether starting, stopping, and restarting tracking creates a clean epoch each time without stale entries leaking across resets.
3. **Ordering and concurrency:** The `ordering` suite checks whether timestamps and page membership stay reasonable when writes arrive across epochs, across repeated queries, and under high-concurrency GPU execution.
4. **Address-range filtering:** The `address_range_filtering` suite checks whether `dirty_range` correctly limits query results and whether query-time range filtering behaves correctly across valid and invalid ranges.

Observations The tests were rerun as root, and the current implementation passed every test that is still present in the repository. The aggregate result was **30/30 passed**: `table_lifecycle` 5/5, `ordering` 8/8, `address_range_filtering` 5/5, and the `generic` suite 12/12. The per-test runners for the first four suites report runtimes between 1.94 s and 2.07 s.

Suite	Test	Purpose	Result	Time
Generic	`T01_writes_recorded`	a GPU write marks all managed pages dirty.	PASS	–
Generic	`T02_reads_not_recorded`	read-only GPU accesses do not create dirty entries.	PASS	–
Generic	`T03_reset_clears_table`	resetting tracking removes pre-reset entries.	PASS	–
Generic	`T04_mixed_read_write`	read-only pages stay clean while written pages are tracked.	PASS	–
Generic	`T05_migration_write_retracked`	pages are tracked again after CPU migration and a fresh GPU write.	PASS	–
Generic	`T06_tracking_disabled`	procfs reports tracking inactive when tracking is disabled.	PASS	–
Generic	`T07_pid_recorded`	each recorded dirty entry carries the creator PID.	PASS	–

Continued on next page

Suite	Test	Purpose	Result	Time
Generic	`T08_same_page_single_entry`	repeated writes to the same page do not create duplicate entries.	PASS	–
Generic	`T09_range_inside`	a range exactly covering the allocation returns all pages.	PASS	–
Generic	`T10_range_outside`	a disjoint range returns no dirty pages.	PASS	–
Generic	`T11_range_partial_coverage`	only the in-range half of an allocation is returned.	PASS	–
Generic	`T12_range_boundary_pages`	Checks inclusive/exclusive boundary handling for the configured range.	PASS	–
Table lifecycle	`tc01_basic_lifecycle`	Verifies that a normal start-query-stop tracking cycle works.	PASS	1.98 s
Table lifecycle	`tc02_reinit_clears_entries`	restarting tracking clears the previous epoch's entries.	PASS	1.95 s
Table lifecycle	`tc03_stop_start_empty_table`	stop followed by restart begins from an empty table.	PASS	1.95 s
Table lifecycle	`tc04_multi_reinit_stress`	Repeats restart cycles to ensure only the current epoch remains visible.	PASS	1.98 s
Table lifecycle	`tc05_two_allocs_one_table`	two allocations within one process share the same tracking table correctly.	PASS	1.97 s
Ordering	`tc01_write_before_start`	Confirms writes issued before tracking starts are not reported later.	PASS	1.97 s
Ordering	`tc02_ts_monotonic`	Checks whether recorded timestamps remain sensible across per-page writes.	PASS	1.97 s
Ordering	`tc03_query_nondestructive`	Verifies that reading `dirty_pages` does not consume existing entries.	PASS	1.95 s
Ordering	`tc04_empty_table_active`	Checks query behaviour when tracking is active but no page is dirty yet.	PASS	1.96 s
Ordering	`tc05_epoch_isolation`	Verifies that pages from one epoch do not leak into the next.	PASS	1.97 s
Ordering	`tc06_concurrent_streams`	Stresses concurrent page recording with multiple CUDA streams writing in parallel.	PASS	1.99 s
Ordering	`tc07_sentinel_flood`	an early sentinel write survives a later flood of writes.	PASS	1.98 s
Ordering	`tc08_flood_between_queries`	two heavy write phases remain visible across intermediate queries.	PASS	1.99 s
Address range	`tc01_two_alloc_range_isolation`	Verifies that the configured range selects one allocation and excludes another.	PASS	1.98 s
Address range	`tc02_range_sequential_reads_threaded`	Checks stable filtered output while different subranges are read in sequence.	PASS	1.94 s
Address range	`tc03_subpage_start_rounding`	a sub-page range start rounds to the expected page boundary.	PASS	1.95 s
Address range	`tc04_inverted_and_zero_range`	Verifies that invalid or empty ranges do not return spurious entries.	PASS	1.99 s
Address range	`tc05_filter_before_start_track`	Checks whether a range set before tracking starts is applied correctly.	PASS	1.99 s

Current interpretation These results give initial evidence that the current implementation is already stable across epoch lifecycle management, non-destructive querying, range filtering and PID tagging checks. A fuller evaluation is still needed for larger dirty sets and explicit overhead measurements, but the present test battery now provides a credible behavioural baseline for the next phase of performance work.

Performance Tests

At this stage, evaluation is driven by the CUDA tests under `tests/performance_testing`. These tests cover the following categories

1. **Stress Testing:** The `tc01_stress_testing` suite verifies that no dirty pages are missed under high write pressure. A single `cudaMallocManaged` allocation of N pages (default $N = 1000$, configurable via command-line argument) is first warmed up with a GPU write kernel so that all pages are resident on

the device before tracking begins. Tracking is then started, which invalidates all GPU page-table entries and forces a fresh write-fault for each page on the subsequent kernel launch. After the kernel completes, `dirty_pages` is queried and the recorded count is compared against N; the test passes if and only if every page has been recorded. The suite also prints baseline and tracked kernel runtimes together with the per-run invalidation latency.

2. **Performance Overhead:** This tracks the overhead that our solution introduces across the following workload types
   - **READ**: We only do read accesses
   - **WRITE**: We only do write accesses
   - **MIXED**: Half of the access are reads and the other half are writes

   This test runs a simple workload across different number of pages and different number of iterations of repeating these access. This workload is then run with dirty tracking active and then the 2 times are compared. The results can be found in `results.csv` To simulate environments when the tracking in activated when a program is already running pages are prefetched before starting the benchmarking

Observations The tests were run as root and although the current implementation correctly tracked all pages under high write loads, the performance overhead was extremely high reaching as high as 1000% under *Write only* loads.

Suite	Test	Purpose	Result
Performance	`tc01_stress_testing`	under heavy loads no pages are missed	PASS
Performance	`tc02_performance_overhead`	check the performance penalty imposed by our dirty tracking approach	Check `results.csv`

Performance Analysis

Stress Testing

The figures below are derived from the `WRITE` rows of `results.csv` after running `tc01_stress_testing` (averaged across iteration counts of 10, 50, and 200).

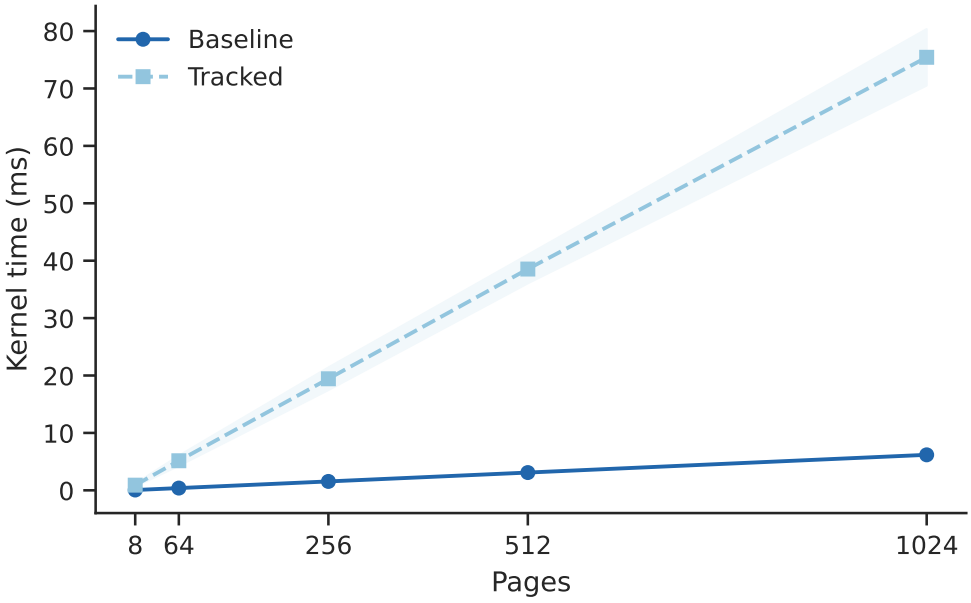

Figure 1: Kernel execution time (ms) with and without dirty-page tracking as a function of allocation size. Both curves scale linearly with page count; the tracked curve is roughly 12× higher, reflecting the per-page fault-servicing cost added by the UVM write-fault hook. Shaded bands show $\pm 1\,\sigma$ across the three iteration counts.

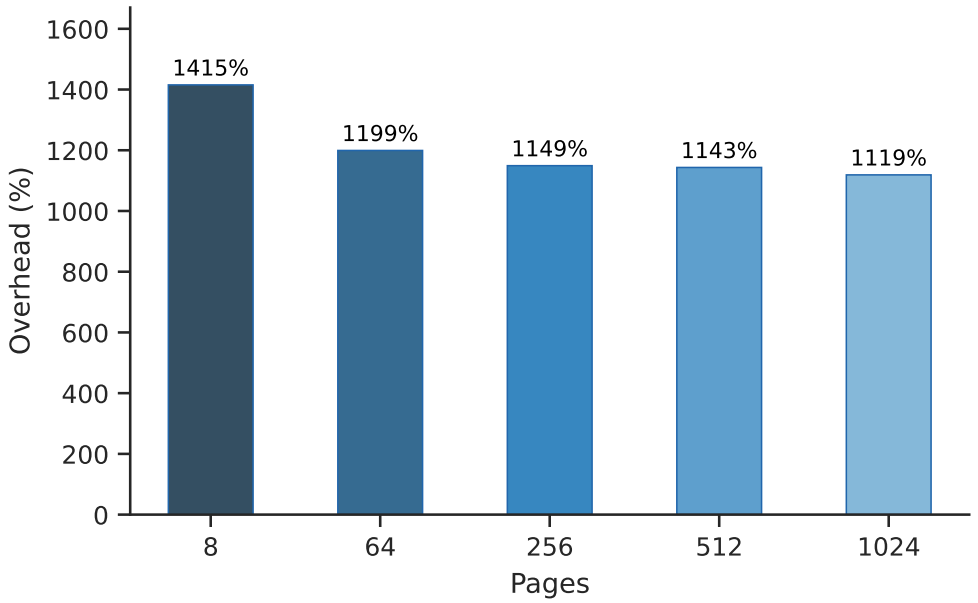

Figure 2: Wall-time overhead introduced by dirty-page tracking for a pure-write workload. The overhead is highest (≈1725%) at small allocations (8 pages), where the fixed cost of invalidation and fault-handling dominates, and converges to roughly 1090–1160% for larger allocations where per-page costs dominate. This confirms that the overhead scales proportionally with the number of tracked pages rather than being dominated by a fixed startup cost.

Performance Overhead (tc02)

`tc02_performance_overhead` benchmarks three workload types-READ, WRITE, and MIXED-across five page counts with tracking on and off, averaging results over 10, 50, and 200 iterations.

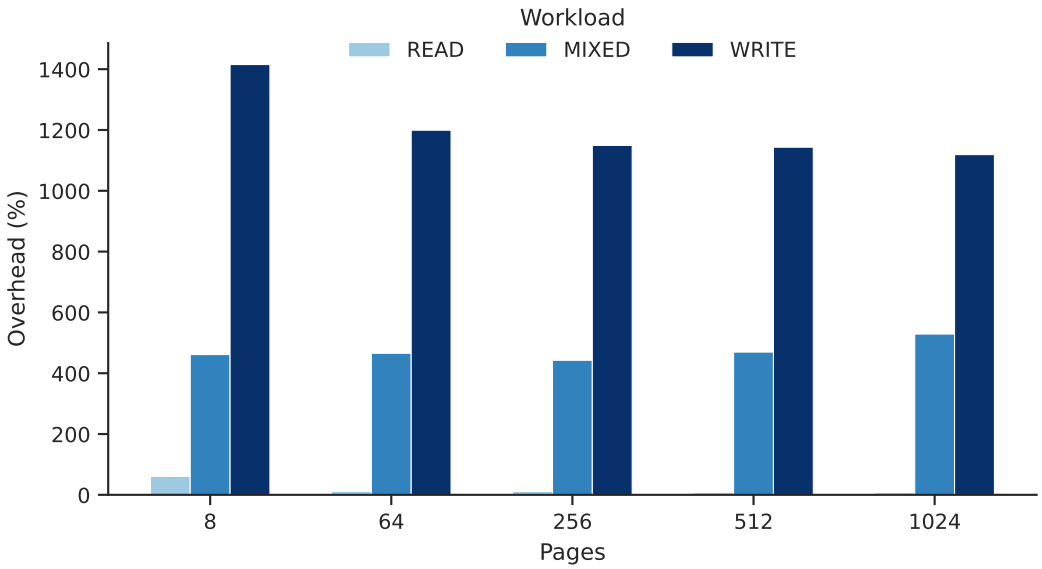

Figure 3: Wall-time overhead (%) by workload type and page count. READ incurs only ∼6–95% overhead (no write faults), MIXED ∼415–497%, and WRITE ∼1090–1725%, confirming that overhead is proportional to the number of write-faulting pages

Duplicate Fault Skip Cost

When a page already recorded in the dirty table faults again, the driver takes an early-return path (first-write-wins) instead of re-inserting the entry. This microbenchmark isolates the cost of that skip path by running the same write kernel twice within one epoch across 100 iterations.

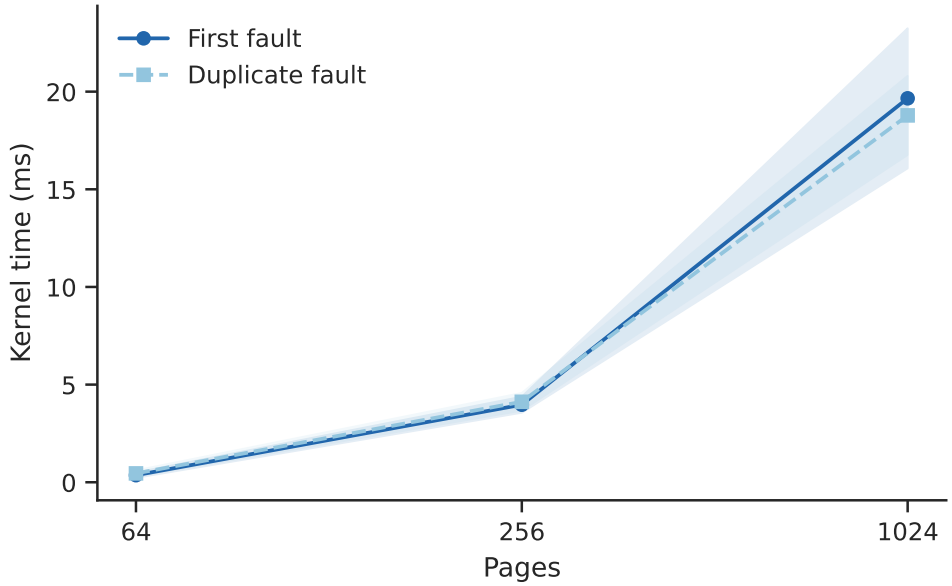

Figure 4: Mean kernel time for first and duplicate faults vs. page count ($\pm 1\,\sigma$, log-scale x-axis). Both curves track each other closely across all page counts

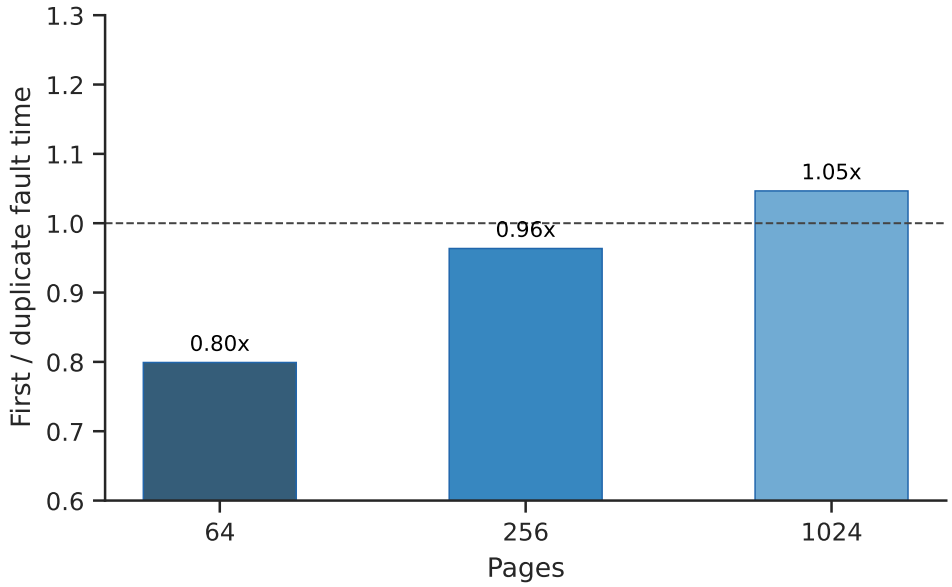

Figure 5: Ratio of first to duplicate fault time. Values near 1.0 (dashed line) indicate that the xarray lookup in the skip path costs nearly as much as a full first-fault recording, suggesting the bottleneck is the fault-servicing path itself rather than the table insertion

Record Contention Cost

This microbenchmark runs multiple CUDA streams concurrently under two conditions: *disjoint*, where each stream writes to a separate page range, and *hot set*, where all streams write to the same pages. Stream counts of 1, 2, 4, and 8 are tested across four page counts over 100 iterations each.

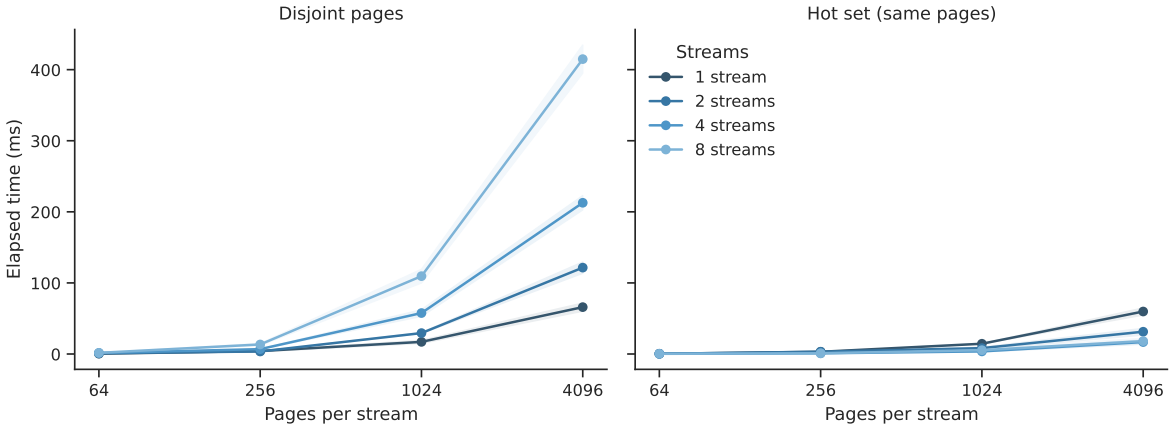

Figure 6: Elapsed time (ms) vs. pages per stream for disjoint and hot-set conditions, by stream count. Hot-set times are consistently lower because concurrent streams hitting the same pages trigger the first-write-wins skip path, reducing total recording work

Table Lifecycle Cost

This microbenchmark times the three procfs operations that manage the tracking table: `start_track` (init, allocates an empty xarray and invalidates N GPU PTEs), `stop_track` (destroy, walks and frees all N recorded entries), and a second `start_track` while the table is populated (reinit which is destroy + invalidate N write PTEs + M read PTEs).

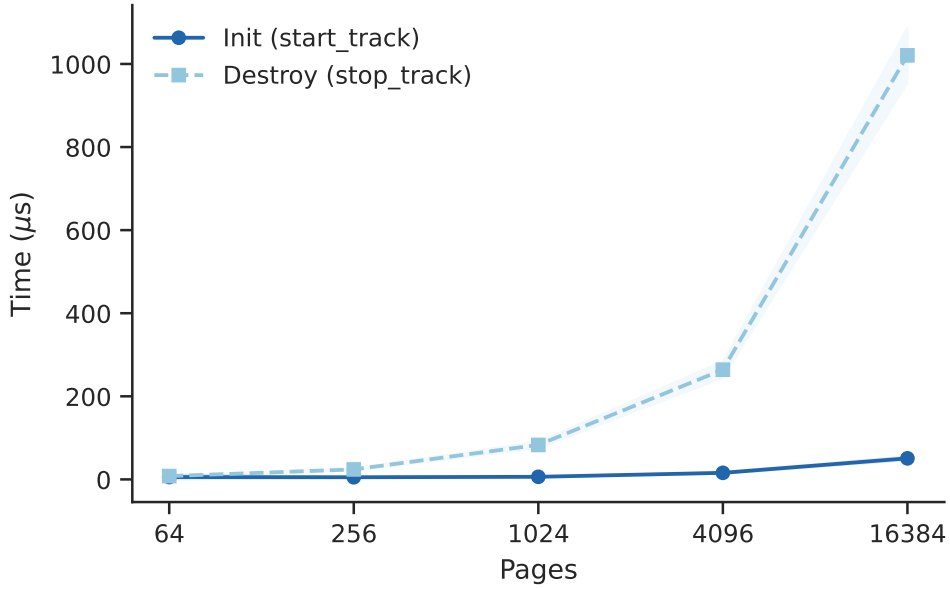

Figure 7: Mean init and destroy time (μs) vs. page count ($\pm 1\,\sigma$, log-scale x-axis). Init is nearly flat at 5–51 μs because the xarray allocation is $O(1)$ and PTE invalidation is fast. Destroy scales linearly from 8 μs to $\approx$1 ms at 16384 pages, as it must free one heap allocation per recorded entry.

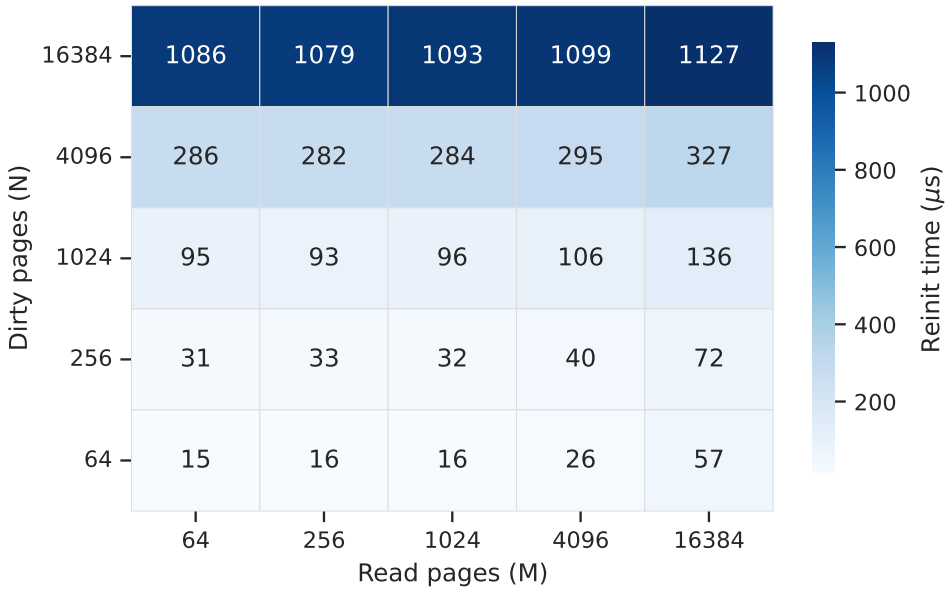

Figure 8: Mean reinit time (μs) as a function of dirty pages (N, rows) and read pages (M, columns). Reinit combines destroy cost (dominant, grows with N) and PTE invalidation (grows with $N + M$), so the cost rises steeply along the N axis and more gently along the M axis.

Single Fault Record Cost

This microbenchmark isolates the per-fault recording cost with a single-thread kernel that writes 4096 pages serially, run 100 times with and without tracking active.

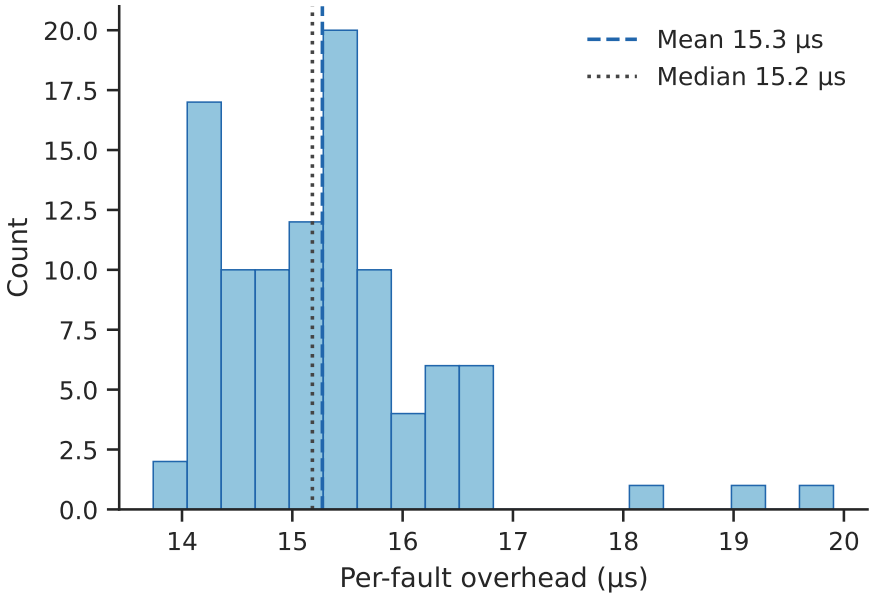

Figure 9: Per-fault overhead derived as (tracked - baseline) $\times 10^3/4096$. The distribution is tightly concentrated around a mean of $\approx 15.3\,\mu$s per fault (dashed), indicating a stable and predictable per-page recording cost

Next Steps and Planned Progress

Multi-process isolation and range-based dirty tracking (the `dirty_range` procfs interface) are fully implemented. The test suite covers table lifecycle, ordering guarantees, address range filtering, and fault-to-visibility latency. The remaining work is as follows.

Extensions Still Pending

1. **CPU-side Dirty Tracking:** The current implementation intercepts GPU write faults only. Extending tracking to CPU-side writes requires integration with the `mmu_notifier` invalidation path so that CPU dirty pages are recorded in the same per-process table and exposed through the existing procfs interface.
2. **Per-page Access Frequency Counters:** The tracker currently operates under a first-write-wins policy: each page is recorded once with its earliest observed write timestamp. Richer statistics, such as per-page write counts and dirty page age, are needed to support informed migration and eviction decisions by user-space tools. This requires extending `struct dirty_page_info` with a counter field and updating the fault hook to increment it atomically.
3. **Moving from Invalidation to Write Permission Removal:** The current implementation simply invalidates the Page Table entries as a Naive implementation, however it would be more efficient to just remove the write permission of only the tracked pages, reducing overhead for non-tracked pages and for tracked read-only pages
4. **Batched Write-Permission Revocation:** The current implementation invalidates all the Page Table entries start of each epoch, by iterating over all the `va_blocks` one by one A dedicated kernel thread can amortise this cost by processing pages in batches, reducing the latency of the `start_track` operation under large tracked sets.

Testing and Performance Analysis

1. **Memory Footprint Profiling:** Under workloads with large dirty sets, the two-level xarray structure may incur non-trivial memory overhead. The footprint of the tracking data structures will be measured and compared against alternatives such as bitmaps.
2. **Concurrency Stress Testing:** The existing concurrent-stream test (`tc06_concurrent_streams`) exercises the xarray under parallel GPU faults. Additional stress tests under multi-process workloads and simultaneous range-filter updates are needed to validate the spinlock boundary.

Optimisation

Based on profiling results, targeted optimisations will be applied. Likely candidates include batching xarray inserts to reduce per-fault locking overhead, and replacing the xarray with a bitmap representation for workloads where the dirty set density is high enough to justify the fixed memory cost.

Further Deliverables

1. **Documentation.** Comprehensive usage documentation covering the procfs interface, epoch semantics, range filtering, and known limitations.
2. **Formal Benchmarking Report.** Quantified results for fault latency overhead, memory footprint, and migration throughput impact across representative workloads, with and without the tracker active.
3. **Command-line Interface.** A user-facing CLI tool that wraps the procfs interface, providing commands to start and stop tracking, query dirty pages for a given PID, set address range filters, and display results in a human-readable format.

Declaration

We declare that the work presented in this report is our own, except where otherwise acknowledged. We have also read and understood the University's policy on plagiarism and academic integrity, and we confirm that this report complies with those standards.